

Szuvenírek

Amaru szuveníreket vásárol egy külföldi boltban, ahol N különböző **típusú** szuvenír van. Minden szuvenírtípusból végtelen sok darab kapható a boltban.

Minden szuvenírtípusnak van egy meghatározott ára. A i -edik ($0 \leq i < N$) típusú szuvenír ára $P[i]$ érme, ahol $P[i]$ egy pozitív egész szám.

Amaru tudja, hogy a szuvenírtípusok az árak szerint csökkenő sorrendben vannak felsorolva, és az áraik különbözőek. Konkrétan: $P[0] > P[1] > \dots > P[N-1] > 0$. Továbbá ismeri a $P[0]$ értékét. Sajnos Amaru semmilyen más információval nem rendelkezik a szuvenírek árairól.

Néhány szuvenír megvásárlásához Amaru több tranzakciót is végrehajt az eladóval.

Minden tranzakció a következő lépésekből áll:

1. Amaru átad bizonyos (pozitív) számú érmét az eladónak.
2. Az eladó ezeket az érméket egy kupacba rakja a hátsó szobában lévő asztalra, ahol Amaru nem láthatja őket.
3. Az eladó minden egyes szuvenírtípust egyenként megvizsgál, a sorszámuk szerint növekvő sorrendben $(0, 1, \dots, N-1)$. Minden típust **pontosan egyszer** vizsgál meg egy tranzakcióban.
 - Az i -edik típus vizsgálatakor, ha a halomban lévő érmék jelenlegi száma legalább $P[i]$, akkor
 - az eladó eltávolít $P[i]$ érmét a halomból, és
 - az eladó egy i -edik típusú szuvenírt tesz az asztalra.
4. Az eladó odaadja Amarunak a kupacban maradt összes érmét és az asztalon lévő összes szuvenírt.

Megjegyezzük, hogy a tranzakció megkezdése előtt nincsenek érme vagy szuvenírek az asztalon.

A feladatod, hogy bizonyos számú tranzakció végrehajtására utasítsd Amarut úgy, hogy:

- minden egyes tranzakció során **legalább** egy szuvenírt vásároljon, és
- összességében minden i -re (ahol $0 \leq i < N$) **pontosan** i darab szuvenírt vásároljon az i -edik típusból. Megjegyezzük, hogy ez azt jelenti, hogy Amaru nem vásárolhat egyetlen 0-adik típusú szuvenírt sem.

Amarunak nem kell minimalizálnia a tranzakciók számát, és korlátlan mennyiségű pénzérme áll a rendelkezésére.

Megvalósítás

A következő eljárást kell elkészítened.

```
void buy_souvenirs(int N, long long P0)
```

- N : a szuvenírtípusok száma.
- $P0$: a $P[0]$ értéke.
- Ez az eljárás minden egyes tesztesetnél pontosan egyszer hívódik meg.

A fenti eljárás a következő függvényt hívhatja meg Amaru utasítván, hogy hajtson végre egy tranzakciót:

```
std::pair<std::vector<int>, long long> transaction(long long M)
```

- M : az Amaru által az eladónak átadott érmék száma.
- A függvény egy párt ad vissza. A pár első eleme az L tömb, amely a megvásárolt szuvenírtípusokat tartalmazza (növekvő sorrendben). A második elem egy R egész szám, az Amaru után visszaadott érmék száma.
- Teljesülnie kell, hogy a $P[0] > M \geq P[N - 1]$. A $P[0] > M$ feltétel biztosítja, hogy Amaru nem vásárol 0 típusú szuvenírt, a $M \geq P[N - 1]$ pedig azt, hogy Amaru legalább egy szuvenírt vásárol. Ha ezek a feltételek nem teljesülnek, a megoldásod a következő hibaüzenetet kapja: `Output isn't correct: Invalid argument`. Vegyük észre, hogy a $P[0]$ -val ellentétben a $P[N - 1]$ értéke nem szerepel a bemenetben.
- A függvény minden egyes tesztesetben legfeljebb 5000 alkalommal hívható meg.

Az értékelő viselkedése **nem adaptív**. Ez azt jelenti, hogy az árak P sorozata már a `buy_souvenirs` meghívása előtt rögzített.

Korlátok

- $2 \leq N \leq 100$
- $1 \leq P[i] \leq 10^{15}$ minden i -re, ahol $0 \leq i < N$.
- $P[i] > P[i + 1]$ minden i -re, ahol $0 \leq i < N - 1$.

Részfeladatok

Részfeladat	Pontszám	További megszorítások
1	4	$N = 2$
2	3	$P[i] = N - i$ minden olyan i -re, ahol $0 \leq i < N$.
3	14	$P[i] \leq P[i + 1] + 2$ minden i -re, ahol $0 \leq i < N - 1$.
4	18	$N = 3$
5	28	$P[i + 1] + P[i + 2] \leq P[i]$ minden i -re, ahol $0 \leq i < N - 2$. $P[i] \leq 2 \cdot P[i + 1]$ minden olyan i -re, ahol $0 \leq i < N - 1$.
6	33	Nincsenek további megszorítások.

Példa

Tekintsük a következő hívást.

```
buy_souvenirs(3, 4)
```

$N = 3$ szuvenírtípus van és $P[0] = 4$. Figyeljük meg, hogy az árak P sorozatára csak három lehetőség van: $[4, 3, 2]$, $[4, 3, 1]$, és $[4, 2, 1]$.

Tegyük fel, hogy a `buy_souvenirs` meghívja a `transaction(2)`-t. Tegyük fel, hogy a hívás a $([2], 1)$ visszatérési értéket adja vissza, ami azt jelenti, hogy Amaru vásárolt egy 2-es típusú szuvenírt, és az eladó visszaadott neki 1 érmét. Figyeljük meg, hogy ebből kikövetkeztethetjük, hogy $P = [4, 3, 1]$, mivel:

- A `transaction(2)` $P = [4, 3, 2]$ esetén a $([2], 0)$ eredményt adta volna vissza.
- A `transaction(2)` $P = [4, 2, 1]$ esetén a $([1], 0)$ eredményt adta volna vissza.

Ezután a `buy_souvenirs` meghívhatja a `transaction(3)`-at, amely az $([1], 0)$ -t adja vissza, ami azt jelenti, hogy Amaru vásárolt egy 1-es típusú szuvenírt, és az eladó visszaadott neki 0 érmét. Amaru eddig összesen vásárolt egy 1-es és egy 2-es típusú szuvenírt.

Végül a `buy_souvenirs` meghívhatja a `transaction(1)`-et, amely a $([2], 0)$ eredménnyel tér vissza, ami azt jelenti, hogy Amaru vásárolt egy 2-es típusú szuvenírt. Megjegyezzük, hogy itt használhattuk volna a `transaction(2)` hívást is. Ezen a ponton Amarunak összesen van egy 1-es típusú és két 2-es típusú szuvenírje, ahogyan az szükséges.

Mintaértékelő

Bemeneti formátum:

N

$P[0] \quad P[1] \quad \dots \quad P[N-1]$

Kimeneti formátum:

$Q[0] \quad Q[1] \quad \dots \quad Q[N-1]$

Itt $Q[i]$ az összesen vásárolt i -edik típusú szuvenírek száma minden i -re, ahol $0 \leq i < N$.